
Lab W09: Sort It Yourself

Crate Expectations

A note on how this assignment was developed

This assignment was drafted with AI help (mostly for brainstorming new problems to solve and formatting with LaTeX). It should be accurate, but something might still be wrong. Because this is a new assignment, you are also helping me debug it, and I will take that into consideration when I review your work. That said, helping me find rough edges is not a substitute for doing the assignment; each problem below is solvable, but you may need to make assumptions in your attempt. If you find an error or something unclear, please tell me; that helps later versions, though finding errors is not the point of this assignment.

1 Overview

This lab asks you to implement a sorting algorithm you have not seen presented or named in this course. You already have a working `Sorts` class from the lectures, holding bubble sort, selection sort, insertion sort, mergesort, and quicksort. You will add one new sorting method to that same class. It is built almost entirely out of one simple operation, described below, plus a helper method you already have.

2 The Scenario

A conveyor belt holds a row of n numbered crates. A robot sits at the far left end of the belt. The robot has exactly one capability: it can reach in, grab a block of crates starting from the left end, the first k crates in the row for any k you choose, and rotate that entire block end over end before setting it back down. This reverses the order of just those k crates. Every crate past position k is left exactly where it was.

Call this operation `reverse(k)`. Using only repeated calls to `reverse`, with whatever value of k you choose each time, you will bring the row of crates into ascending order.

How `reverse` works

Suppose the row currently holds six crates:

4, 2, 6, 1, 5, 3

Calling `reverse(3)` grabs the first three crates and reverses that block. The last three crates, position 4 and on, are untouched.

[4, 2, 6], 1, 5, 3 $\longrightarrow$ [6, 2, 4], 1, 5, 3

The row is now 6, 2, 4, 1, 5, 3.

Calling `reverse(5)` on this new row grabs the first five crates and reverses that block. The last crate, position 6, is untouched.

$[6, 2, 4, 1, 5], 3 \longrightarrow [5, 1, 4, 2, 6], 3$

The row is now 5, 1, 4, 2, 6, 3.

Notice that `reverse(k)` only ever touches a prefix of the row. Nothing past position k ever moves, and the robot cannot reach into the middle of the row or grab crates out of order; it can only take the first k as a block and turn that block around.

3 Your Task

Add the following two methods to your `Sorts` class.

First, a helper that performs exactly the operation described above:

```
// Java
private static <T extends Comparable<? super T>> void
    reverse(T[] theArray, int k)

// C++
template <typename T>
static void reverse(T theArray[], int k)
```

This reverses `theArray[0..k-1]` in place. Everything from index k onward is left untouched.

Second, the sorting method itself:

```
// Java
public static <T extends Comparable<? super T>> void
    beltSort(T[] theArray)

// C++
template <typename T>
static void beltSort(T theArray[], int n)
```

This should sort `theArray` into ascending order using only calls to your new `reverse` method. You may not swap two items directly, and you may not rearrange the array in any other way; every change to the array's order has to happen through a call to `reverse(k)`, the same restriction the robot is under.

Hint: the `indexOfLargest` method can be useful.

Before writing any code, work out your strategy with handwritten notes, on paper or a whiteboard, not typed. Trace a small example, six or seven crates is enough, the same way you traced the other sorts earlier in this unit.

Annotate your trace as you go. For each step, write down which `reverse(k)` call you are making, the value of k you chose, and the row that results, the same way Section 2.1 labels each call in the worked example above. A bare sequence of numbers with no labels does not show your process and will not satisfy this requirement.

This is not optional practice; take a clear photo or screenshot of your annotated handwritten work and submit it. I want to see how you developed the solution, not just the pseudocode or the finished method; your notes should be clear and complete enough that I can follow your reasoning step by step without having to guess at it.

4 Project Setup and Submission

You have been given a folder named labw09 with the project already set up: for Java, this is an Eclipse project with the package structure already in place; for C/C++, this is the source layout together with the Makefile. Do your work inside this folder as given. Do not rename it or rearrange its structure, the layout is part of what gets submitted.

Your **Sorts** class already contains the other five sorts from this unit. Add **reverse** and **beltSort** to this same class, following the existing style and generic bound used by every other method in it.

Your **Sorter** class already demonstrates each sort on both an **Integer** array and a **Droid** array. Add a demonstration of **beltSort** in the same style, sorting both kinds of arrays, below the point marked in the file.

If you are working in Java, submit the entire labw09 Eclipse project, not just the individual .java files.

If you are working in C/C++, submit the entire labw09 folder, including the Makefile and the rest of the project layout, not just the Sorts header file on its own.

- A short comment at the top of **beltSort** (three or four sentences is plenty) explaining, in your own words, the strategy your method uses.
- Photos of your handwritten development and annotated trace